

# Carga de *datasets*

## Modelos Avanzados de Aprendizaje Automático I

En esta asignatura se utilizarán distintos modelos avanzados de Aprendizaje Automático para resolver problemas del mundo real. De forma previa al uso de estos modelos, y sin necesidad de avanzar contenidos teóricos, en este ejercicio se propone la carga y preprocesado básico de los *datasets* que se van a utilizar. Estos *datasets* se proveerán en un archivo comprimido, y se pueden dividir en 2 grupos:

- *Datasets* relativos al repositorio PMLB (*Penn Machine Learning Benchmarks*). Este repositorio es una colección de *datasets* estandarizados utilizados para evaluar algoritmos de Aprendizaje Automático. PMLB ofrece una amplia variedad de *datasets* de diferentes dominios y tamaños, que son adecuados tanto para problemas de clasificación como de regresión, con más de 160 *datasets* provenientes de diversas fuentes, lo que permite evaluar algoritmos en una amplia gama de contextos. Además, PMLB está diseñado para integrarse fácilmente con herramientas y bibliotecas como Scikit-Learn o MLJ, proporcionando una interfaz sencilla para cargar y utilizar los *datasets*. Debido a esto, este repositorio permite realizar evaluaciones sistemáticas y comparaciones entre diferentes algoritmos y configuraciones, promoviendo la investigación reproducible en el campo del Aprendizaje Automático. Para hacer las prácticas de esta asignatura, se proveen 30 *datasets* relacionados con problemas de clasificación en 2 clases, y 30 *datasets* relacionados con problemas de regresión. La URL desde donde se puede acceder al repositorio es <https://epistasislab.github.io/pmlb/>
- *Dataset* relativo al tipo de aprendizaje denominado *Stream Learning* o aprendizaje en flujo de datos. Concretamente, el *dataset* a utilizar se llama *Electricity*, y es uno de los conjuntos de datos más utilizados en el campo del aprendizaje en flujo y el aprendizaje incremental. Este dataset se originó a partir del mercado de la electricidad en Nueva Gales del Sur, Australia, y contiene datos horarios de precios de la electricidad y demanda, así como otros factores relacionados, en un mercado competitivo. Cada registro en el conjunto de datos representa la situación en un punto en el tiempo específico y se utiliza para predecir si el precio de la electricidad en una determinada hora será más alto o más bajo que el precio de la hora anterior. Este *dataset* se puede descargar de la siguiente dirección: <https://github.com/vlosing/driftDatasets/tree/master/realWorld/Elec2>

Las variables independientes de este *dataset* son las siguientes:

- *date*: Fecha y hora del registro.

- *day*: Día de la semana (números naturales entre 1 y 7)
- *period*: Periodo del día (1-48, para intervalos de media hora).
- *nswprice*: Precio de la electricidad en Nueva Gales del Sur.
- *nswdemand*: Demanda de electricidad en Nueva Gales del Sur.
- *vicprice*: Precio de la electricidad en Victoria.
- *vicdemand*: Demanda de electricidad en Victoria.
- *transfer*: Cantidad de electricidad transferida entre Nueva Gales del Sur y Victoria.

En el artículo original, se recomienda eliminar los atributos 1 (*date*) y 4 (*nswprice*). El resto de atributos, excepto *day*, están normalizados. Este atributo *day* tiene valores cíclicos que tienen que ser codificados como se muestra más adelante.

Por su parte, la variable objetivo, *class*, indica si el precio de la electricidad es más alto (UP) o más bajo (DOWN) en comparación con la hora anterior. Este *dataset* se provee como dos archivos:

- *elec2\_data.dat*, con los atributos como una matriz de tamaño 45312x8.
- *elec2\_label.dat*, con las etiquetas, como una matriz de tamaño 45312x1.

Para cargar estos datasets, es necesario antes importar (*using*) los siguientes paquetes en Julia: *FileIO*, *Images*, *JLD2*.

Como se muestra, el *dataset* relativo a *Stream Learning* contiene un atributo con valores cíclicos, en este caso relativos a días de la semana, codificados como números naturales del 1 al 7. En un caso de tener pocos valores posibles, como es el caso, una opción habitual es realizar un *one-hot-encoding*. Sin embargo, otras veces el número de valores distintos puede ser mucho mayor (como el número de días en un año), o incluso ser un valor continuo, por lo que realizar la codificación *one-hot-encoding* no resulta práctica. En este ejercicio, se realizará la codificación habitual cuando el número de valores cíclicos es alto, o bien son valores continuos. Para ello, es necesario distinguir dos casos distintos, según si los valores cíclicos son discretos o continuos. Un ejemplo del primer caso, con valores discretos, está en el atributo a preprocesar en este *dataset*: los días de la semana. Otros ejemplos típicos podrían ser las estaciones, los días del año, o las horas del día tomando únicamente la hora, sin fracción de minutos/segundos. Un ejemplo del segundo caso, con valores continuos, podría ser la hora del día, pero tomando la fracción de minutos/segundos. En general, cualquier atributo cíclico en el que se tenga un número muy alto de valores podría ser considerado como continuo. La diferencia entre ambos tipos es que, en el caso de valores discretos, el último valor no equivale al primero, y en el caso de valores continuos esto sí que pasaría, por lo que esta situación tiene que ser considerada dentro de la formulación. Por ejemplo, en los días de la semana el día 7 no equivale al día 1; sin embargo, en la hora del día, el instante 23:59:59.99 sí equivale al instante

00:00:00 del día siguiente (a no ser que se necesite una precisión extremadamente alta).

En general, para realizar la codificación, se hace primero una normalización entre máximo y mínimo de los valores, puesto que están acotados, pasando a un intervalo no entre 0 y 1, sino entre 0 y  $2\pi$ , y después se construyen dos nuevos atributos, con los senos y los cosenos de estos ángulos. En el caso de atributos continuos, la circunferencia se rellenará casi en su totalidad, mientras que en el caso de los atributos discretos se tomarán tantos ángulos equiespaciados como valores se tengan, siendo el último valor no igual a  $2\pi$  (esto conllevaría que el último sea igual al primero).

Para hallar estos valores de ángulos, para el caso continuo, la normalización y transformación en ángulo se puede hacer mediante la ecuación

$$rad = 2\pi \frac{x - min}{max - min}$$

donde x es el valor del atributo. En esta ecuación, la parte  $\frac{x-min}{max-min}$  transforma x para pasarlo al intervalo [0,1], y, al multiplicarlo por  $2\pi$  se convierte en valores entre 0 y  $2\pi$ .

Sin embargo, como se ha dicho, para valores discretos esto no sería correcto, puesto que con esta ecuación el valor máximo coincidiría con el ángulo  $2\pi$ , que es igual al ángulo 0, es decir, el primer valor. Para ello, una solución sencilla es aumentar el intervalo en el que están los valores. Por ejemplo, los días de la semana, en lugar de estar entre 1 y 7, se considera que el valor máximo es el 8. De esta forma, el valor 7 no ocupará el ángulo 0, que tiene el primer valor. En general, para hacer esto es necesario calcular la magnitud de cambio entre valores contiguos de este atributo. Para los días de la semana sería un valor de 1, pero este valor podría ser distinto, por ejemplo, si las 4 estaciones del año se tienen como [0, 0.25, 0.5, 0.75], este valor sería igual a 0.25. Una vez que se tiene este valor, la ecuación sería muy similar a la anterior:

$$rad = 2\pi \frac{x - min}{max - min + m}$$

donde m sería este valor de cambio. Estas dos ecuaciones para estos dos casos (continuo y discreto) se pueden unir en una sola, igual a la anterior, teniendo en cuenta que  $m=0$  para el caso continuo.

En este ejercicio, se da la siguiente hecha la función *intervalDiscreteVector*, que permite calcular el valor de m de la ecuación anterior relativa a la codificación de datos cíclicos. Recibe un vector de valores reales, e intenta calcular el valor de cambio entre los valores. Si son valores discretos, devuelve este valor de cambio. Si son valores continuos, devuelve un valor de 0. Con esta función, las dos ecuaciones anteriores para codificar valores cíclicos, continuos y discretos, se pueden unir en una sola de forma muy sencilla.

En este ejercicio, se pide realizar las siguientes funciones:

- *fileNamesFolder*. Devuelve los nombres de archivos que hay en una carpeta. Recibe:
  - *folderName*, de tipo *String*, con el nombre de una carpeta a examinar.
  - *extension*, de tipo *String*, con la extensión que tienen los nombres de archivos a buscar, sin el punto.

Esta función debe devolver un vector de valores de tipo *String*, con los nombres de los archivos encontrados en esa carpeta, pero con los nombres sin la extensión.

Para desarrollar esta función puede ser útil utilizar las funciones *isdir*, *readdir*, *uppercase*, *filter*. Una forma sencilla de tomar los nombres de archivo de una carpeta que tengan una determinada extensión puede ser haciendo lo siguiente:

```
extension = uppercase(extension);  
fileNames = filter(f -> endsWith(uppercase(f), ".$extension"), readdir(folderName))
```

De esta forma, sólo sería necesario eliminar la extensión de los nombres de los archivos.

En el desarrollo de esta función no se permite el uso de bucles.

- *loadDataset*. A partir del nombre del *dataset* y de la carpeta donde está, carga el *dataset* correspondiente y lo devuelve. Recibe como parámetros obligatorios:
  - *datasetName*, de tipo *String*, con el nombre del *dataset* sin la extensión. En general, en los *datasets* tomados del repositorio PMLB, el nombre del *dataset* será el nombre del archivo sin la extensión. Por tanto, el primer paso será añadir la extensión “.tsv” al nombre del archivo.
  - *datasetFolder*, de tipo *String*, con el nombre de la carpeta donde está el *dataset*.

Como parámetro opcional, tiene:

- *datasetType*, de tipo *DataType*, con el tipo de dato a cargar para las entradas. Por defecto tiene como valor *Float64*.

Esta función cargará el *dataset* correspondiente y lo devolverá como una tupla (*inputs*, *targets*), donde *inputs* será una matriz de valores del tipo especificado en *datasetType* y *targets* será un vector de valores booleanos o reales con la salida deseada, según si el *dataset*

se refiere a un problema de clasificación (valores booleanos, en este caso todos los *datasets* que se proveen son de 2 clases) o de regresión. Si no se encuentra el archivo en cuestión, esta función devolverá *nothing*.

Algunas funciones que pueden ser útiles para desarrollar esta función son *joinpath*, *abspath*, *readdlm* (esta está dentro del paquete *DelimitedFiles*), *findall*, *unique*, *setdiff*.

Para cargar el archivo, se puede usar la función *readdlm*, especificando el tabulador como separador con `'\t'`. Esto devolverá una matriz, donde la primera fila serán los encabezados. Lo primero que hay que hacer es hallar qué columna se corresponde con la salida deseada, será aquella cuyo encabezado sea igual a "target". Una vez hallada esta columna, se toman las entradas y las salidas deseadas de la siguiente forma:

- Entradas: filas de la segunda a la última, y todas las columnas excluyendo la de las salidas deseadas. Será necesario convertir estos valores al tipo especificado en la llamada.
- Salidas deseadas: Si estas toman únicamente valores en el conjunto {0,1}, este será un problema de clasificación, por lo que se convertirán a valores booleanos. En otro caso, se convertirán al tipo indicado en *datasetType*.

Para el desarrollo de esta función no se permite el uso de bucles.

- *datasetsDescription*. A partir del nombre una carpeta donde está, devuelve una matriz con la descripción de los *datasets* de esa carpeta, referidos a problemas de clasificación o regresión. Recibe como parámetros:

- *datasetFolder*, de tipo *String*, con el nombre de la carpeta donde estarán ubicados los *datasets*.
- *classification*, de tipo *Bool*, que indica si se quiere leer los *datasets* de clasificación o de regresión.

Esta función hará llamadas a *fileNamesFolder* y *loadDataset* para cargar los *datasets*, y posteriormente, según el valor de *classification*, los filtrará y se quedará únicamente con los de clasificación o regresión. Después, debe devolver una matriz con una fila por *dataset*, y 3 columnas, donde la primera será el nombre del *dataset* (sin la extensión), la segunda será el número de instancias, y la tercera el número de atributos. Esta matriz deberá estar ordenada de forma creciente por el tamaño del *dataset*, entendiendo tamaño como número de

instancias multiplicado por el número de atributos. Para el desarrollo de esta función no se permite el uso de bucles.

- *cyclicalEncoding*. Dado un vector de valores reales, interpreta estos datos como cíclicos, y realiza su codificación como senos y cosenos. Recibe como parámetro obligatorio:
  - *data*, de tipo *AbstractArray{<:Real,1}*, con los valores del atributo a codificar.

Funciones que pueden ser útiles: *maximum*, *minimum*, *extrema*, *intervalDiscreteVector* (esta se provee en este ejercicio), *sin*, *cos*. Esta función se apoya en una llamada a la función *intervalDiscreteVector*, que devuelve el valor de *m* de la ecuación anterior, que puede tener un valor igual a 0, para aplicar la ecuación anterior. Una vez aplicada, solamente queda realizar el cálculo de los senos y los cosenos de estos ángulos.

Esta función debe devolver una tupla con 2 vectores: senos y cosenos (por este orden) resultados de realizar esta codificación.

Para el desarrollo de esta función no se permite el uso de bucles.

- *loadStreamLearningDataset*. Devuelve el *dataset* utilizado para resolver el problema de *Stream Learning*. Recibe como parámetro obligatorio:
  - *datasetFolder*, de tipo *String*, con la carpeta donde están los archivos "elec2\_data.dat" y "elec2\_label.dat"

Como parámetro opcional, tiene:

- *datasetType*, de tipo *DataType*, con el tipo de dato a devolver para las entradas.

Funciones que pueden ser útiles: *joinpath*, *abspath*, *readdlm*, *vec*, *setdiff*, *cyclicalEncoding* (esta se desarrolla en este ejercicio).

Este *dataset* se provee como dos archivos, de nombres "elec2\_data.dat" y "elec2\_label.dat", con las entradas y salidas deseadas respectivamente. Ambos archivos se pueden cargar con la función *readdlm*. Las salidas deseadas convertirán a booleanos, y posteriormente a vector con *vec*. En cambio, en las entradas hay que hacer más preprocesado:

- Eliminar las columnas 1 (*date*) y 4 (*nswprice*).
- Llamar a *cyclicalEncoding* con los valores de la primera columna (día) resultante de la operación anterior.
- Concatenar las salidas de senos y cosenos, por este orden, a la matriz resultante de eliminar la primera columna con el día, es decir, ponerlas como las primeras

columnas de la matriz de entradas a devolver.

Esta función deberá devolver una tupla con 2 elementos, por este orden:

- Entradas, como una matriz bidimensional con valores del tipo especificado. Esta matriz tendrá 7 columnas, con los valores, por este orden, de senos, cosenos, *period*, *nswdemand*, *vicprice*, *vicdemand* y *transfer*.
- Salidas deseadas, como un vector de valores booleanos.

Para el desarrollo de esta función no se permite el uso de bucles.

---

### Firmas de las funciones:

A continuación se muestran las firmas de las funciones a realizar en este ejercicio. Tened en cuenta que, dependiendo de cómo se defina la función, esta puede contener o no la palabra reservada *function* al principio:

```
function fileNameFolder(folderName::String, extension::String)
function loadDataset(datasetName::String, datasetFolder::String; datasetType::DataType=Float64)
function datasetsDescription(datasetFolder::String, classification::Bool)
function cyclicalEncoding(data::AbstractArray{<:Real,1})
function loadStreamLearningDataset(datasetFolder::String; datasetType::DataType=Float64)
```